

Chapter 5

Transitioning to Convolutional Neural Networks (CNNs)

Objective: To understand the limitations of fully connected neural networks (FCNNs) for image data and introduce the basic concepts of Convolutional Neural Networks (CNNs). This chapter will focus on exploring the architecture and advantages of CNNs.

1. Why Move to Convolutional Neural Networks?

FCNNs treat each pixel independently, ignoring the spatial relationships in images. CNNs address this limitation by leveraging the spatial structure of image data, making them far more effective for image classification tasks.

Tasks:

1. Compare FCNNs and CNNs for image classification:
 - How do FCNNs process an image after flattening? What spatial information might be lost?
 - How do CNNs maintain spatial relationships between pixels?
2. Research and list the main advantages of CNNs over FCNNs for image data.

2. Key Components of CNNs

CNNs consist of several specialized layers designed to process image data efficiently. Below are some of the critical layers:

2.1 Convolutional Layers

Convolutional layers apply filters (small matrices) to the input image to extract features like edges, textures, and shapes.

- **Tasks:**

1. Research how convolutional filters work. What is the purpose of sliding a filter across an image?
2. What is a **feature map**? Why does its size depend on parameters like filter size, stride, and padding?

Questions to Explore:

- What features might the first convolutional layer detect compared to deeper layers?
- Why do CNNs use multiple filters in each layer?

1.2 Pooling Layers

Pooling layers down-sample the feature maps, reducing their size while retaining essential features. The most common type is **max pooling**, which takes the maximum value from each region.

- **Tasks:**

1. Research the difference between max pooling and average pooling.
2. Why is down-sampling important? How might it help with overfitting?

Questions to Explore:

- What is the effect of pooling on computational efficiency?
- Could pooling layers cause a loss of information? Why or why not?

1.3 Fully Connected Layers in CNNs

CNNs typically end with fully connected layers to combine extracted features and make predictions.

- **Tasks:**

1. Why do CNNs still need fully connected layers for classification?
2. How does the input to the fully connected layers differ from the input to the convolutional layers?

3. CNN Architecture Basics

A basic CNN for image classification might look like this:

1. **Convolutional Layer:** Extracts features from the image.
2. **Pooling Layer:** Reduces feature map dimensions.
3. **Convolutional Layer:** Extracts more complex features.
4. **Pooling Layer:** Further reduces feature map dimensions.
5. **Flatten Layer:** Converts the feature maps into a vector.
6. **Fully Connected Layers:** Makes the final classification.

Tasks:

1. Design a simple CNN architecture for CIFAR-10:
 - How many convolutional and pooling layers should you include?
 - How many filters will each convolutional layer use?
 - What activation function should you use in the convolutional layers?
2. Sketch your architecture and label the dimensions of the data at each layer.

4. Build Your First CNN (Programming Assignment)

Tasks:

1. Define a CNN architecture using **TensorFlow/Keras Sequential API**.
2. Include at least:
 - One convolutional layer with ReLU activation.
 - One pooling layer.
 - A fully connected layer with softmax activation.
3. Compile the model with the same loss function and optimizer used in Session 3.

Skeleton Code to Guide Your Implementation:

```
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Conv2D, MaxPooling2D, Flatten, Dense

# Define a simple CNN
model = Sequential([
    Conv2D(32, (3, 3), activation='relu', input_shape=(32, 32, 3)), # Convolutional layer
    MaxPooling2D(pool_size=(2, 2)), # Pooling layer
    Conv2D(64, (3, 3), activation='relu'), # Second convolutional layer
    MaxPooling2D(pool_size=(2, 2)), # Second pooling layer
    Flatten(), # Flatten the feature maps
    Dense(128, activation='relu'), # Fully connected layer
    Dense(10, activation='softmax') # Output layer for 10 classes
])

# Display the model summary
model.summary()
```

5. Reflection

After defining your CNN, reflect on the following:

Questions to Reflect On:

1. How does the number of parameters in your CNN compare to your FCNN from Chapter 3? Why are CNNs typically more efficient?
2. What challenges did you face when deciding the architecture?
3. What improvements might you make to your CNN in future iterations?